



Experiment 05: Containerizing Applications using Docker

Experiment No.	05
Mapped CO	CO4
Aim	To containerize an application by writing a Dockerfile, building an image and running it as a container.
Tools / Software	Docker Engine; A sample web application such as Python Flask or Node.js; Docker Hub account optional; Code editor / terminal

Theory

A container is an isolated and runnable instance of an application packaged with its required dependencies. Containers share the host operating-system kernel and therefore start faster and consume fewer resources than traditional virtual machines. Docker is a commonly used platform for building, running and managing containers.

A Docker image is an immutable, layered template created from instructions written in a Dockerfile. Common Dockerfile instructions include FROM, WORKDIR, COPY, RUN, EXPOSE and CMD. When an image is executed, it becomes a container. Images can be tagged and stored in a registry such as Docker Hub for sharing and deployment.

Procedure

1. Install Docker Engine and verify the installation using `docker --version`.
2. Create a project directory containing a sample web application file and a dependency file such as `requirements.txt`.
3. Write a Dockerfile defining the base image, working directory, dependency installation, port exposure and start command.
4. Build the Docker image using `docker build` and assign a meaningful tag such as `myapp:1.0`.
5. Run the image as a container using `docker run` and map the container port to a host port.
6. List running containers using `docker ps` and inspect container logs using `docker logs`.
7. Open the application in a browser or use `curl` to verify that it is reachable on the mapped port.
8. Optionally tag and push the image to Docker Hub for sharing.

Sample Code / Commands

app.py

```
from flask import Flask
app = Flask(__name__)

@app.route('/')
def home():
    return "Hello from Docker Container"

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

requirements.txt

```
flask
```



PARSHWANATH CHARITABLE TRUST'S
A.P. SHAH INSTITUTE OF TECHNOLOGY
Department of Computer Science and Engineering
Data Science



Dockerfile

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 5000
CMD ["python", "app.py"]
```

Build, run and publish commands

```
docker --version
docker build -t myapp:1.0 .
docker run -d -p 5000:5000 --name myapp myapp:1.0
docker ps
docker logs myapp
curl http://localhost:5000
docker tag myapp:1.0 <user>/myapp:1.0
docker push <user>/myapp:1.0
```

Observation / Experiment Output

Instructions to students: Paste screenshots or copied outputs in the boxes below after completing Docker installation verification, application creation, image building, container execution and output validation. Keep screenshots clear and readable.

Name	Roll No.	Batch	Date
Vighnesh Pingale	48	B2	14/08/26

1. Docker Installation and Version Output

```
apsit@apsit-HP-Pro-Tower-280-G9-PCI-Desktop-PC:~$ docker --version
Docker version 28.1.1, build 4eba377
apsit@apsit-HP-Pro-Tower-280-G9-PCI-Desktop-PC:~$ docker info
Client: Docker Engine - Community
Version: 28.1.1
Context: default
Debug Mode: false
Plugins:
  buildx: Docker Buildx (Docker Inc.)
    Version: v0.23.0
    Path: /usr/libexec/docker/cli-plugins/docker-buildx
  compose: Docker Compose (Docker Inc.)
    Version: v2.35.1
    Path: /usr/libexec/docker/cli-plugins/docker-compose
Server:
Containers: 0
Running: 0
Paused: 0
Stopped: 0
Images: 2
Server Version: 28.1.1
Storage Driver: overlay2
Backing Filesystem: extfs
Supports d_type: true
Using metacopy: false
Native Overlay Diff: true
userxattr: false
Logging Driver: json-file
Cgroup Driver: systemd
Cgroup Version: 2
Plugins:
  Volume: local
  Network: bridge host ipvlan macvlan null overlay
  Log: awslogs fluentd gcplogs gelf journald json-file local splunk syslog
Swarm: inactive
Runtimes: io.containerd.runc.v2 runc
Default Runtime: runc
Init Binary: docker-init
containerd version: 05044ec0a9a75232cad458027ca83437aee3f4da
```

```
apsit@apsit-HP-Pro-Tower-280-G9-PCI-Desktop-PC:~$ docker info
Storage Driver: overlay2
Backing Filesystem: extfs
Supports d_type: true
Using metacopy: false
Native Overlay Diff: true
userxattr: false
Logging Driver: json-file
Cgroup Driver: systemd
Cgroup Version: 2
Plugins:
  Volume: local
  Network: bridge host ipvlan macvlan null overlay
  Log: awslogs fluentd gcplogs gelf journald json-file local splunk syslog
Swarm: inactive
Runtimes: io.containerd.runc.v2 runc
Default Runtime: runc
Init Binary: docker-init
containerd version: 05044ec0a9a75232cad458027ca83437aee3f4da
runc version: v1.2.5-0-g59923ef
Init version: de40ad0
Security Options:
  apparmor
  seccomp
  Profile: builtin
  cgroupns
Kernel Version: 6.8.0-136-generic
Operating System: Ubuntu 22.04.5 LTS
OSType: linux
Architecture: x86_64
CPUs: 12
Total Memory: 7.45GiB
Name: apsit-HP-Pro-Tower-280-G9-PCI-Desktop-PC
ID: 86a8622b-c713-4b0d-b775-a7d7ddf954c8
Docker Root Dir: /var/lib/docker
Debug Mode: false
Experimental: false
Insecure Registries:
  ::1/128
  127.0.0.0/8
Live Restore Enabled: false
apsit@apsit-HP-Pro-Tower-280-G9-PCI-Desktop-PC:~$
```



2. Application Source and Dependency Files Output

```
CPUs: 12
Total Memory: 7.45GiB
Name: apsit-HP-Pro-Tower-280-G9-PCI-Desktop-PC
ID: 86a8622b-c713-4b0d-b775-a7d7ddf954c8
Docker Root Dir: /var/lib/docker
Debug Mode: false
Experimental: false
Insecure Registries:
::1/128
127.0.0.0/8
Live Restore Enabled: false

apsit@apsit-HP-Pro-Tower-280-G9-PCI-Desktop-PC:~$ mkdir -p ~/docker-flask-demo
cd ~/docker-flask-demo
apsit@apsit-HP-Pro-Tower-280-G9-PCI-Desktop-PC:~/docker-flask-demo$ pwd
/home/apsit/docker-flask-demo
apsit@apsit-HP-Pro-Tower-280-G9-PCI-Desktop-PC:~/docker-flask-demo$ ls -la
total 16
drwxrwxr-x 2 apsit docker 4096 Aug 14 09:16 .
drwxr-x--- 67 apsit apsit 12288 Aug 14 09:16 ..
apsit@apsit-HP-Pro-Tower-280-G9-PCI-Desktop-PC:~/docker-flask-demo$ nano app.py
apsit@apsit-HP-Pro-Tower-280-G9-PCI-Desktop-PC:~/docker-flask-demo$ nano requirements.txt
xt
apsit@apsit-HP-Pro-Tower-280-G9-PCI-Desktop-PC:~/docker-flask-demo$ ls -l
total 8
-rw-rw-r-- 1 apsit docker 184 Aug 14 09:22 app.py
-rw-rw-r-- 1 apsit docker 6 Aug 14 09:23 requirements.txt
apsit@apsit-HP-Pro-Tower-280-G9-PCI-Desktop-PC:~/docker-flask-demo$ cat app.py
from flask import Flask

app = Flask(__name__)

@app.route('/')
def home():
    return "Hello from Docker Container"

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
apsit@apsit-HP-Pro-Tower-280-G9-PCI-Desktop-PC:~/docker-flask-demo$ cat requirements.txt
flask
apsit@apsit-HP-Pro-Tower-280-G9-PCI-Desktop-PC:~/docker-flask-demo$ |
```



PARSHWANATH CHARITABLE TRUST'S

A.P. SHAH INSTITUTE OF TECHNOLOGY

Department of Computer Science and Engineering
Data Science



3. Dockerfile Output

```
apsit@apsit-HP-Pro-Tower-280-G9-PCI-Desktop-PC:~/docker-flask-demo$ nano Dockerfile
apsit@apsit-HP-Pro-Tower-280-G9-PCI-Desktop-PC:~/docker-flask-demo$ cat Dockerfile
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 5000
CMD ["python", "app.py"]
apsit@apsit-HP-Pro-Tower-280-G9-PCI-Desktop-PC:~/docker-flask-demo$ ls -l
total 12
-rw-rw-r-- 1 apsit docker 184 Aug 14 09:22 app.py
-rw-rw-r-- 1 apsit docker 156 Aug 14 09:25 Dockerfile
-rw-rw-r-- 1 apsit docker 6 Aug 14 09:23 requirements.txt
apsit@apsit-HP-Pro-Tower-280-G9-PCI-Desktop-PC:~/docker-flask-demo$
```

4. Docker Image Build Output

```
apsit@apsit-HP-Pro-Tower-280-G9-PCI-Desktop-PC:~/docker-flask-demo$ docker build -t myapp:1.0 .
[+] Building 42.5s (10/10) FINISHED                                docker:default
=> [internal] load build definition from Dockerfile                0.1s
=> == transferring dockerfile: 195B                               0.0s
=> [internal] load metadata for docker.io/library/python:3.12-slim 17.2s
=> [internal] load .dockerignore                                  0.0s
=> == transferring context: 2B                                     0.0s
=> [1/5] FROM docker.io/library/python:3.12-slim@sha256:dd29372629eeba2dd003fd 11.4s
=> == resolve docker.io/library/python:3.12-slim@sha256:dd29372629eeba2dd003fd9 0.1s
=> == sha256:013c5454f914638dc2bfcab71c0630de0089f7ebff0a7239b5 5.0kB / 5.0kB 0.0s
=> == sha256:26c307b5e35a59ce911f5fde5b9458120ec8734e831ea2d 29.70MB / 29.70MB 10.5s
=> == sha256:fa18dfb1257a9c1afc75e233c55b0195dd02b0d0d18dd7e24c 1.29MB / 1.29MB 1.0s
=> == sha256:b952fe9f0810de5dac5d24a3983aee5f3e3804f7359a533b 12.12MB / 12.12MB 4.7s
=> == sha256:dd29372629eeba2dd003fd9e9d35a5b8236c44727075a036 10.37kB / 10.37kB 0.0s
=> == sha256:876416ecd9aca2bec90a1fb0c7a9500bbf749f5700b70f02d 1.75kB / 1.75kB 0.0s
=> == sha256:07e0bfe2ff00c4530bc73b2f88a1e9615a50c9a77028f41f0bb4b9 240B / 240B 1.3s
=> == extracting sha256:26c307b5e35a59ce911f5fde5b9458120ec8734e831ea2da5649a9a 0.4s
=> == extracting sha256:fa18dfb1257a9c1afc75e233c55b0195dd02b0d0d18dd7e24c10230 0.0s
=> == extracting sha256:b952fe9f0810de5dac5d24a3983aee5f3e3804f7359a533b325e40 0.2s
=> == extracting sha256:07e0bfe2ff00c4530bc73b2f88a1e9615a50c9a77028f41f0bb4b97 0.0s
=> [internal] load build context                                  0.1s
=> == transferring context: 487B                                   0.0s
=> [2/5] WORKDIR /app                                           0.1s
=> [3/5] COPY requirements.txt .                                0.1s
=> [4/5] RUN pip install --no-cache-dir -r requirements.txt     13.3s
=> [5/5] COPY . .                                              0.1s
=> == exporting to image                                         0.1s
=> == exporting layers                                           0.1s
=> == writing image sha256:3311241781439118a07e4f84f6a76d966a5a1f6072cd0ad8067b 0.0s
=> == naming to docker.io/library/myapp:1.0                     0.0s
apsit@apsit-HP-Pro-Tower-280-G9-PCI-Desktop-PC:~/docker-flask-demo$
```

```
apsit@apsit-HP-Pro-Tower-280-G9-PCI-Desktop-PC:~/docker-flask-demo$ docker images
REPOSITORY    TAG       IMAGE ID       CREATED        SIZE
myapp         1.0       331124178143  2 minutes ago  132MB
ubuntu        latest    f794f40ddfff  5 months ago  78.1MB
hello-world   latest    1b44b5a3e06a  12 months ago  10.1kB
apsit@apsit-HP-Pro-Tower-280-G9-PCI-Desktop-PC:~/docker-flask-demo$
```



5. Container Run, Logs and Application Access Output

```
apsit@apsit-HP-Pro-Tower-280-G9-PCI-Desktop-PC:~/docker-flask-demo$ docker ps
docker logs myapp
curl http://localhost:5000
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS
ca069fa8809e   myapp:1.0      "python app.py"         2 minutes ago  Up 2 minutes  0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp
myapp
* Serving Flask app 'app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a
production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://172.17.0.2:5000
Press CTRL+C to quit
172.17.0.1 - - [14/Aug/2026 04:04:16] "GET / HTTP/1.1" 200 -
Hello from Docker Containerapsit@apsit-HP-Pro-Tower-280-G9-PCI-Desktop-PC:~/docker-flas
k-demo$ |
```

6. Docker Hub Push Output (Optional)

```
Hello from Docker Containerapsit@apsit-HP-Pro-Tower-280-G9-PCI-Desktop-PC:~/docker-flas
k-demo$ docker stop myapp
myapp
apsit@apsit-HP-Pro-Tower-280-G9-PCI-Desktop-PC:~/docker-flask-demo$ docker rm myapp
myapp
apsit@apsit-HP-Pro-Tower-280-G9-PCI-Desktop-PC:~/docker-flask-demo$ docker ps -a
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS        NAMES
apsit@apsit-HP-Pro-Tower-280-G9-PCI-Desktop-PC:~/docker-flask-demo$ |
```



PARSHWANATH CHARITABLE TRUST'S
A.P. SHAH INSTITUTE OF TECHNOLOGY
Department of Computer Science and Engineering
Data Science



Final Observation Statement:

The Flask web application was successfully containerized using Docker by creating a Dockerfile and building the `myapp:1.0` image. The container was run with port mapping, and the application was successfully accessed using `curl` on port 5000. Container logs were verified, and the container was stopped and removed successfully.

Expected Output

The Docker image should build successfully, the container should run in the background, the application should be reachable on the mapped host port, and `docker ps` should list the running container. If Docker Hub is used, the tagged image should be visible in the repository.

Conclusion

The application was containerized by writing a Dockerfile, building a Docker image and running it as an isolated portable container, demonstrating image creation, container execution, port mapping and basic container lifecycle management.

Viva / Review Questions

1. Differentiate a container from a virtual machine.
2. What is the difference between a Docker image and a Docker container?
3. Explain the purpose of EXPOSE in a Dockerfile and the `-p` flag in `docker run`.
4. Why are Docker image layers cached?

Marks: ____ / 10

Date: _____ Signature of Faculty: _____